Phishio: Detecção Colaborativa de Phishing em Tempo Real via Abordagem Híbrida de Recuperação de Informação e *Crowdsourcing*[*]

Phishio: Real-Time Collaborative Phishing Detection through a Hybrid Information Retrieval and Crowdsourcing Approach

Pedro Henrique Gomes Lückeroth[1]
Lucas dos Santos Lima[2]
Sinaide Nunes Bezerra[3]

Resumo

O *phishing* persiste como uma das ameaças cibernéticas mais críticas ao explorar a confiança do usuário para o roubo de credenciais. As defesas tradicionais, baseadas em listas negras estáticas, são ineficazes contra a rápida rotatividade de URLs maliciosas em ataques *zero-hour*. Este trabalho propõe o desenvolvimento de um sistema de detecção em tempo real, composto por uma extensão de navegador e uma API de análise em nuvem. A metodologia adota uma abordagem híbrida que combina técnicas de Recuperação de Informação em especial o Modelo Vetorial com ponderação TF-IDF e Similaridade do Cosseno aplicada a uma base de dados proprietária, integrada a um módulo de validação colaborativa (*crowdsourcing*). A arquitetura utiliza o Google Compute Engine para o processamento vetorial intensivo e o Google Firestore para o gerenciamento dinâmico da reputação de URLs baseada no consenso da comunidade. A solução proposta visa reduzir a latência na identificação de novas ameaças e minimizar a taxa de falsos positivos através do *feedback* contínuo dos usuários.

Palavras-chave: Phishing. Recuperação de Informação. Crowdsourcing. Segurança Web. Detecção em Tempo Real.

[*]Trabalho de conclusão de curso, Sistemas de Informação, Unidade São Gabriel

[1]Sistemas de Informação, Brasil– 1213227@sga.pucminas.br
[2]Sistemas de Informação, Brasil– lucas.lima.1129309@sga.pucminas.br
[3]Instituto de Ciências Exatas e de Informática da PUC Minas, Brasil– sinaide@pucminas.br

Abstract

Phishing remains one of the most critical cyber threats, exploiting user trust to steal credentials. Traditional defenses based on static blacklists are ineffective against the rapid turnover of malicious URLs in zero-hour attacks. This work proposes the development of a real-time detection system composed of a browser extension and a cloud-based analysis API. The methodology adopts a hybrid approach combining Information Retrieval techniques, specifically the Vector Space Model with TF-IDF weighting and Cosine Similarity, applied to a proprietary database, integrated with a collaborative validation module (crowdsourcing). The architecture utilizes Google Compute Engine for intensive vector processing and Google Firestore for dynamic URL reputation management based on community consensus. The proposed solution aims to reduce latency in identifying new threats and minimize false positive rates through continuous user feedback.

Keywords: Phishing. Information Retrieval. Crowdsourcing. Web Security. Real-Time Detection.

1 INTRODUÇÃO

A relevância deste estudo é justificada pelo impacto financeiro e operacional dos ataques de engenharia social. Segundo o relatório *Cost of a Data Breach* da IBM (2025), o *phishing* figura de maneira consistente como um dos vetores de acesso inicial mais comuns e custosos para violações de dados corporativos. Corroborando esse cenário, o Federal Bureau of Investigation (2024) aponta o *phishing* como o crime cibernético líder em número de vítimas, gerando perdas bilionárias anualmente.

Apesar do contínuo aprimoramento dos sistemas de detecção, a eficácia dos ataques reside na velocidade. O relatório da Verizon (2024) revela que o tempo mediano para que um usuário seja comprometido após abrir um e-mail de *phishing* é inferior a 60 segundos. Essa janela crítica expõe a fragilidade das listas de bloqueio (*blacklists*) estáticas, que não conseguem acompanhar a volatilidade das ameaças.

Adicionalmente, os atacantes têm migrado suas operações para infraestruturas legítimas para evadir detecções baseadas em reputação de rede. Dados compilados por Martínez (2025), baseados em inteligência da Netcraft, indicam que cerca de 77% dos sites de *phishing* utilizam serviços de nuvem pública para se hospedar. Essa tática, combinada com o ciclo de vida extremamente curto das páginas fraudulentas, muitas vezes ativas por poucas horas, conforme a análise realizada por Kolesnikov e Cuozzo (2021), exige a implementação de mecanismos de detecção em tempo real baseados em análise de conteúdo.

Uma vez que a interação do usuário com páginas fraudulentas ocorre muitas vezes através do navegador web, a integração de ferramentas de segurança diretamente nesse ambiente é estratégica. A consolidação do padrão *Manifest V3* para extensões do Google Chrome corrobora essa premissa ao exigir arquiteturas mais seguras, com foco na privacidade do usuário e no baixo consumo de recursos computacionais.

Diante desse contexto, o objetivo principal deste trabalho é desenvolver uma solução híbrida para a detecção de *phishing* em tempo real. Para alcançar esse propósito, são definidos como objetivos específicos: construir a interface da solução em uma extensão de navegador; implementar uma API em nuvem capaz de realizar o processamento vetorial de textos; e integrar um mecanismo de validação colaborativa (*crowdsourcing*), capaz de unir a capacidade analítica do algoritmo com a inteligência coletiva dos usuários.

A arquitetura proposta assume o caráter de uma solução híbrida devido à convergência de duas frentes distintas de análise. A primeira vertente utiliza o rigor matemático da Recuperação de Informação, por meio do Modelo Vetorial desenvolvido por Baeza-Yates *et al.* (1999), para o processamento técnico do conteúdo das páginas. A segunda vertente estabelece uma camada social baseada no *crowdsourcing*, na qual o consenso e o *feedback* dos usuários refinam a classificação final. Essa integração busca mitigar as limitações de sistemas puramente algorítmicos e reduzir a dependência de listas de bloqueio estáticas.

2 REFERENCIAL TEÓRICO

A fundamentação deste trabalho articula conceitos de segurança da informação, recuperação de informação e usabilidade, estabelecendo a base para uma solução de detecção híbrida. Para a devida compreensão desses pilares. Inicialmente, a exposição do panorama das ameaças de *phishing*, que evidência a evolução técnica dos ataques e as limitações das defesas convencionais. Em seguida, a descrição dos preceitos da Recuperação de Informação e do Modelo Vetorial, a base matemática adotada para a identificação de similaridades textuais. Por fim, a discussão sobre a aplicação do *crowdsourcing* atrelado a princípios de usabilidade justifica a viabilidade de inserir o usuário como agente ativo na validação de segurança.

2.1 O Cenário de Ameaças de Phishing

Segundo a APWG (2025), "o *phishing* é um crime que emprega tanto engenharia social quanto subterfúgios técnicos para roubar dados de identidade pessoal e credenciais de contas financeiras dos consumidores". A evolução dessa ameaça tem sido marcada pela velocidade e pela evasão de defesas convencionais.

Relatórios recentes da indústria indicam que a "janela de oportunidade"para a detecção é crítica. A Verizon (2024) aponta que o tempo mediano para o comprometimento de um usuário é inferior a 60 segundos. Esse cenário é agravado pela volatilidade das URLs: Kolesnikov e Cuozzo (2021) demonstra que grande parte das páginas de *phishing* são descartáveis e permanecem ativas apenas pelo tempo necessário para capturar um número específico de vítimas, o que torna ineficazes as abordagens baseadas em listas negras (*blacklists*) que possuem latência de atualização.

Além disso, a infraestrutura dos ataques mudou. Dados compilados por Martínez (2025) revelam que atacantes utilizam massivamente serviços de nuvem legítimos para hospedar páginas fraudulentas, o que dificulta bloqueios baseados na reputação de IP ou domínio.

Fette, Sadeh e Tomasic (2007) estabeleceram, em estudo seminal, as características estruturais (*features*) que permitem a identificação automatizada de *phishing*, como a idade do domínio, a quantidade de pontos na URL e a presença de formulários inseguros. Estas heurísticas são vitais para a modelagem de classificadores modernos.

2.2 Recuperação de Informação e Modelo Vetorial

Para analisar o conteúdo das páginas em tempo real, este trabalho utiliza conceitos de Recuperação de Informação (RI). Segundo Baeza-Yates *et al.* (1999), o Modelo Vetorial permite representar documentos e consultas como vetores em um espaço multidimensional.

Neste modelo, a importância de um termo é calculada através do esquema de ponderação

TF-IDF (*Term Frequency-Inverse Document Frequency*), que penaliza termos muito comuns (como artigos e preposições) e valoriza termos raros e discriminantes. A similaridade entre uma página suspeita e um documento conhecido (assinatura de phishing) é calculada pelo cosseno do ângulo entre seus vetores:

$$sim(d_j, q) = \frac{\vec{d_j} \cdot \vec{q}}{|\vec{d_j}| \times |\vec{q}|} \tag{1}$$

Fonte: Baeza-Yates *et al.* (1999, p. 27).

Zhang, Hong e Cranor (2007) validaram a aplicação dessa teoria na segurança com o sistema CANTINA, ao demonstrar que o conteúdo textual de uma página, processado via TF-IDF, é um forte indicador de sua legitimidade ou intenção maliciosa.

2.3 Crowdsourcing e Usabilidade na Segurança

A abordagem colaborativa, ou *crowdsourcing*, permite utilizar a inteligência coletiva para refinar sistemas de detecção. Em segurança, isso cria um mecanismo de "humano no ciclo"(*human-in-the-loop*), no qual usuários experientes podem sinalizar falsos positivos que algoritmos puramente matemáticos não conseguem discernir.

Para que essa colaboração seja efetiva, a interface deve seguir princípios de usabilidade. Nielsen (1994) preconiza que o sistema deve manter o usuário informado sobre seu estado e permitir a recuperação de erros. No contexto de uma extensão de segurança, isso significa que alertas devem ser claros e o mecanismo de denúncia deve exigir esforço cognitivo mínimo, de forma a evitar a interrupção desnecessária da navegação.

3 METODOLOGIA

A presente pesquisa é de natureza aplicada, uma vez que visa resolver um problema prático de segurança da informação por meio do desenvolvimento de um artefato de software (extensão de navegador e API). Quanto à abordagem do problema, o estudo é quantitativo, visto que utiliza métricas numéricas para avaliar o desempenho do sistema, tais como a latência de processamento e a taxa de acurácia na classificação de URLs (mensurada via matriz de confusão). Do ponto de vista dos procedimentos técnicos, o trabalho consiste em uma pesquisa experimental, visto que o artefato está submetido a testes em um ambiente controlado, que utiliza uma base mista de URLs legítimas e maliciosas para validar as hipóteses de detecção em tempo real. A metodologia de desenvolvimento segue uma abordagem iterativa, dividida nas etapas de construção do *dataset*, implementação da arquitetura em nuvem e desenvolvimento do módulo cliente.

Esta seção aborda, inicialmente, a arquitetura do sistema proposto, que estabelece os três módulos centrais da solução. Na sequência, o fluxo de validação híbrida é detalhado para

ilustrar a interação e o comportamento dinâmico entre esses componentes. Posteriormente, os mecanismos de instrumentação e coleta de dados são descritos; esses mecanismos servem para a mensuração das métricas de desempenho e acurácia. Por fim, os procedimentos metodológicos são listados para organizar o desenvolvimento e a validação experimental em etapas sequenciais e estruturadas.

3.1 Arquitetura do Sistema Proposto

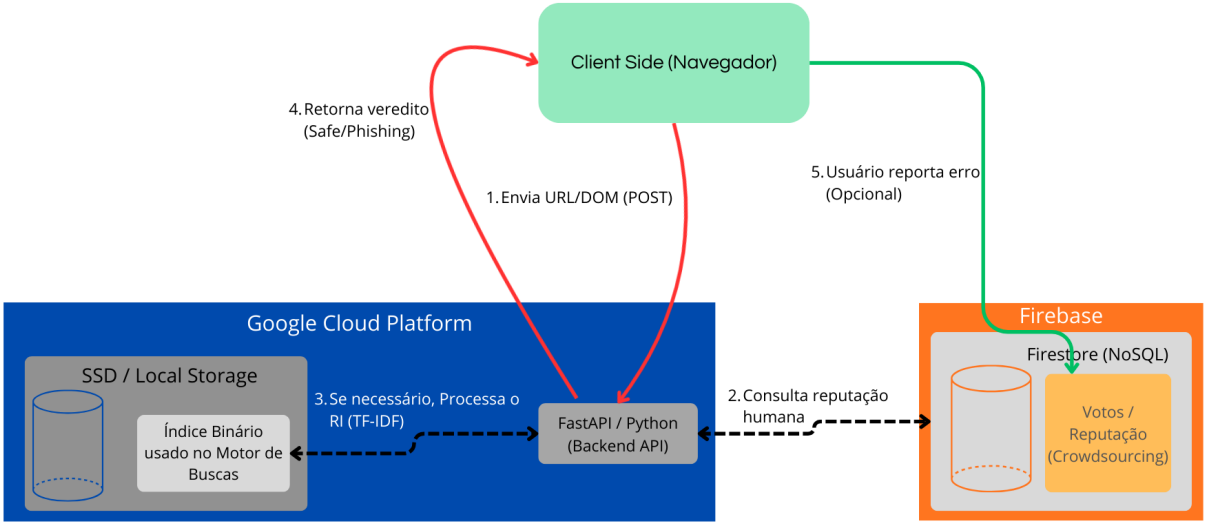
A infraestrutura tecnológica do projeto detalha o modelo de comunicação e a distribuição das responsabilidades computacionais. A proposta mostra como a arquitetura integra a interface do usuário, o processamento analítico em nuvem e o banco de dados colaborativo, o que possibilita a detecção em tempo real.

A solução mostrada na Figura 1 é estruturada em uma arquitetura Cliente-Servidor distribuída, composta por três módulos principais:

1. **Módulo Cliente (Extensão de Navegador):** Desenvolvido em JavaScript conforme proposto pelo padrão *Manifest V3* do Google Chrome. Este módulo opera em segundo plano e intercepta eventos de navegação. Ele é responsável por:
    - Extrair a URL e o conteúdo textual (DOM) da aba ativa;
    - Se comunicar de maneira assíncrona com a API de análise;
    - Exibir alertas visuais na interface do navegador;
    - Coletar o *feedback* do usuário (reporte de falso positivo/negativo).
2. **Módulo de Processamento (Backend API):** Uma API RESTful desenvolvida em Python, hospedada em uma instância virtual no Google Compute Engine (IaaS). Este módulo encapsula o motor de busca vetorial implementado no semestre anterior durante a disciplina de Recuperação de Informação na Web e Redes Sociais (RIWRS[4]). Ele recebe o conteúdo da página, processa os termos (tokenização e *stemming*), calcula o vetor TF-IDF e o compara com a base de assinaturas de *phishing* utilizando a Similaridade do Cosseno.
3. **Módulo de Persistência e Colaboração:** Para gerenciar os dados colaborativos, o Google Firestore é utilizado, consistindo em um banco de dados NoSQL hospedado em nuvem. Ele armazena os votos de reputação enviados pelos usuários. Um algoritmo de consenso é implementado para promover URLs para uma *whitelist* dinâmica quando um limiar de confiança for atingido.

[4]Recuperação de Informação na Web e Redes Sociais.

Figura 1 – Visão arquitetural do módulo cliente, processamento vetorial e base colaborativa



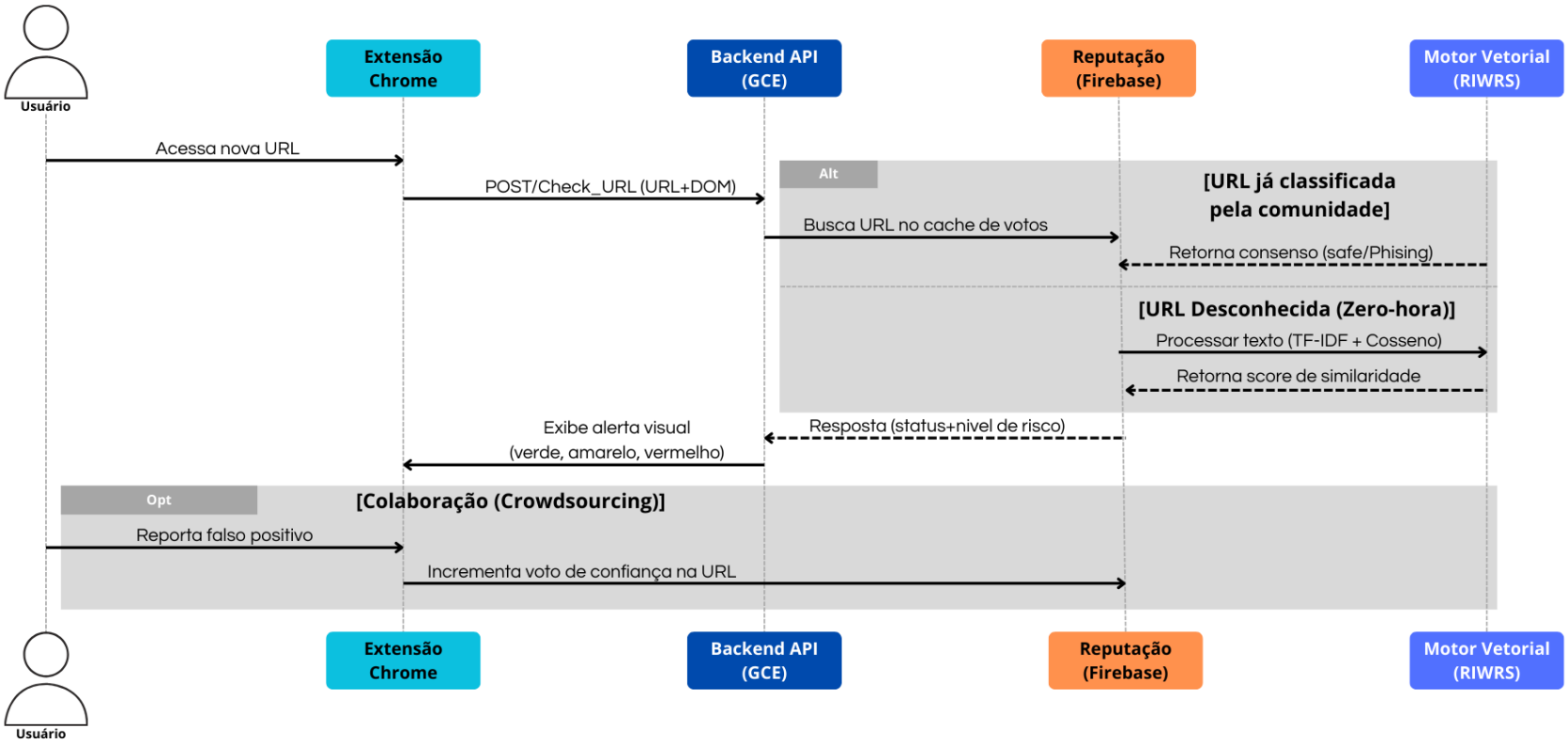
Fonte: Dados da Pesquisa (2026).

3.2 Fluxo de Validação Híbrida

Para compreender a interação dinâmica entre os módulos descritos na arquitetura, a Figura 2 apresenta o diagrama de sequência do sistema. O fluxo foi projetado para otimizar o tempo de resposta e poupar recursos computacionais, fato que fundamenta a natureza híbrida da solução.

Quando o usuário acessa uma nova URL, a extensão intercepta a requisição e envia os dados para a API. Antes de acionar o processamento vetorial (que possui maior custo computacional), o sistema realiza uma verificação no cache de votos colaborativos (Firestore). Se a URL já possuir um consenso formado pela comunidade, o veredito é retornado imediatamente. Apenas em casos de URLs desconhecidas (potenciais ataques *zero-hour*) o motor de Recuperação de Informação é instanciado para calcular a Similaridade do Cosseno com base no texto. Por fim, o ciclo se encerra com a possibilidade de o usuário atuar ativamente ao reportar possíveis falsos positivos para atualizar novamente a base de reputação.

Figura 2 – Diagrama de sequência do fluxo híbrido de validação



Fonte: Dados da Pesquisa (2026).

3.3 Instrumentação e Coleta de Dados

Os mecanismos de monitoramento e armazenamento de dados integrados à solução visam explicitar como o retorno dos usuários e a precisão do algoritmo serão capturados para embasar a avaliação do experimento. O sistema é instrumentado em três níveis de coleta de modo a garantir a observabilidade da eficácia técnica e do impacto social:

- **Métricas de Eficácia e Colaboração (Backend):** O sistema utiliza o *Google Firestore* para o registo da percentagem de ligações identificadas pela comunidade e a relação entre reportes confirmados e falsos positivos.
- **Engajamento e Escala:** O monitoramento da volumetria de usuários ativos é correlacionado ao crescimento da base de dados. Adicionalmente, a localização relativa via IP permite a análise da concentração geográfica das ameaças identificadas
- **Acurácia e Performance Indireta:** O sistema mantém um registro anônimo das URLs classificadas pelo motor vetorial para o cálculo da matriz de confusão. A performance é verificada via ferramentas de inspeção do navegador (*DevTools*), com foco no tempo de resposta das chamadas à API

3.4 Procedimentos Metodológicos

Neste tópico, o roteiro de execução da pesquisa empírica é delineado. O objetivo é fornecer uma visão cronológica de como a construção do artefato e a validação do sistema ocorrerão, desde a preparação da base de dados inicial até a apuração dos resultados. Diante disso, o desenvolvimento segue as seguintes etapas:

- **Etapa 1 - Preparação do Corpus (Cold Start):** Utilização do *dataset* de 100.000 documentos coletados previamente para treinar o índice invertido inicial e gerar os vetores de referência dos sites de *phishing*.
- **Etapa 2 - Desenvolvimento da API:** Migração do código de busca local para um serviço web escalável, otimizando o carregamento do índice, superior a 4,5 GB, na memória RAM da instância em nuvem.
- **Etapa 3 - Integração Híbrida:** Implementação da lógica que consulta primeiro a base colaborativa (Firestore) e, em caso de ausência de registro, aciona o motor vetorial (Compute Engine), para possibilitar performance.
- **Etapa 4 - Validação Experimental:** Realização de testes de acurácia utilizando uma base de controle mista (URLs legítimas e maliciosas) e medição da latência média de resposta da API para garantir que não haja degradação perceptível na navegação do usuário (meta < 500ms).

4 DESENVOLVIMENTO DO MÓDULO CLIENTE

A implementação da extensão de navegador possui base nos padrões modernos de segurança e desempenho exigidos pelo ecossistema do *Google Chrome*. Tal abordagem busca evidenciar as escolhas arquiteturais que viabilizam a detecção contínua sem comprometer a experiência do utilizador. No que concerne à fundamentação sobre o *Manifest V3*, a lista de verificação de migração disponibilizada pelo *Chrome for Developers* (Google, 2024) serviu como principal referência técnica.

4.1 Adoção do Padrão Manifest V3

A escolha pelo Manifest V3 não é apenas uma conformidade técnica, mas uma decisão estratégica para garantir a longevidade e segurança do artefato. O V3, versão mais recente do manifesto da Google, introduz mudanças críticas que permitem uma performance melhor no navegador e fornece maior segurança ao usuário.

- ***Service Workers* (*Background Scripts*):** A extensão utiliza *service_workers* em substituição às páginas de fundo persistentes, o que otimiza o uso de recursos do sistema, e executa apenas quando eventos específicos de navegação ocorrem.
- **Segurança e Privacidade:** O modelo restringe a execução de códigos remotos (como *eval()*), o que exige que toda a lógica de segurança esteja contida no pacote ou seja consultada via APIs seguras (HTTPS), protegendo o usuário de possíveis injeções maliciosas na própria extensão.
- **Permissões Declarativas:** O uso de permissões granulares no *manifest.json* (como *tabs* e *storage*) garante que a extensão acesse apenas os dados estritamente necessários para a análise de *phising*.

4.2 Arquitetura do *Front-end* e Interface do Usuário

O *front-end* da extensão foi projetado seguindo heurísticas de usabilidade de Nielsen, com foco em fornecer *feedback* imediato sem interromper o fluxo de trabalho do usuário. (NIELSEN, 1994)

- **Interceptação e Extração:** O módulo opera de forma assíncrona, extraindo a URL e o conteúdo textual (DOM) da aba ativa assim que o carregamento é concluído.
- **Alertas Visuais:** Caso a API de *backend* retorne um veredito positivo para *phishing*, a interface exibe alertas visuais claros, informando o estado de risco do site.

- **Módulo de *Crowdsourcing (Feedback)*:** A interface inclui um mecanismo de "um clique"para que o usuário possa reportar erros (falsos positivos ou negativos). Esse feedback é enviado ao Google Firestore, atualizando o banco de dados colaborativo e refinar a acurácia do sistema em tempo real.

- **Desempenho (Latência):** A meta de design é que a interação entre o *front-end* e o *back-end* ocorra em menos de 500ms, para garantir que a proteção seja imperceptível na experiência de navegação do usuário.

5 DESENVOLVIMENTO BACKEND

5.1 Validação da Infraestrutura de Processamento e Migração de Dados

Conforme delineado na arquitetura do sistema, o núcleo do motor de busca vetorial foi previamente desenvolvido em ambiente local. Nesta etapa atual da pesquisa, o esforço de engenharia concentrou-se estritamente na preparação da infraestrutura em nuvem, especificamente na migração da base de dados local para o *Google Firestore*.

Adicionalmente, foi implementada uma restrição dimensional heurística no algoritmo de ingestão para garantir a viabilidade técnica no ecossistema NoSQL: termos cujas listas de adjacência ultrapassavam 15.000 registros de documentos (postings) foram categorizados como excessivamente densos e suprimidos do *upload*. Esta decisão baseia-se no fato de que termos com altíssima frequência em um corpus de 100.000 documentos tendem a ser *stop-words* ou vocábulos genéricos, possuindo baixo poder discriminatório para o cálculo da Similaridade do Cosseno. O Trecho de Código 3 demonstra a implementação desta condição na rotina do consumidor (*worker*).

Para contornar essas limitações e otimizar o tempo de *upload*, desenvolveu-se um script de migração dedicado em *Python*, utilizando processamento assíncrono para distribuir a carga de inserção de dados no banco. Adicionalmente, foi implementada uma restrição dimensional heurística no algoritmo de ingestão: termos cujas listas de adjacência ultrapassavam 15.000 documentos (*postings*) foram categorizados como excessivamente densos e suprimidos do *upload*. O Trecho de Código 3 demonstra a simplicidade e eficácia dessa condição implementada na rotina do consumidor (*worker*).

Figura 3 – Implementação do filtro heurístico para termos com alta densidade no Firestore

```
63        # Filtro de segurança para evitar documentos > 1MB no Firestore
64        if num_postings > 15000:
65            status = "PULADO_PESADO"
66            proc_time = (time.time() - start_task) * 1000
67            with csv_lock:
68                with open(METRICS_FILE, 'a', newline='') as f:
69                    csv.writer(f).writerow([termo_limpo, num_postings, proc_time, 0, 0, status])
70            continue
```

Fonte: Dados da Pesquisa (2026).

Essa estratégia de corte não apenas impediu falhas nas requisições por excesso de carga útil, mas também reduziu drasticamente a volumetria transacionada e o tempo total de processamento, descartando vocábulos genéricos que pouco contribuiriam para a capacidade discriminatória do Modelo Vetorial.

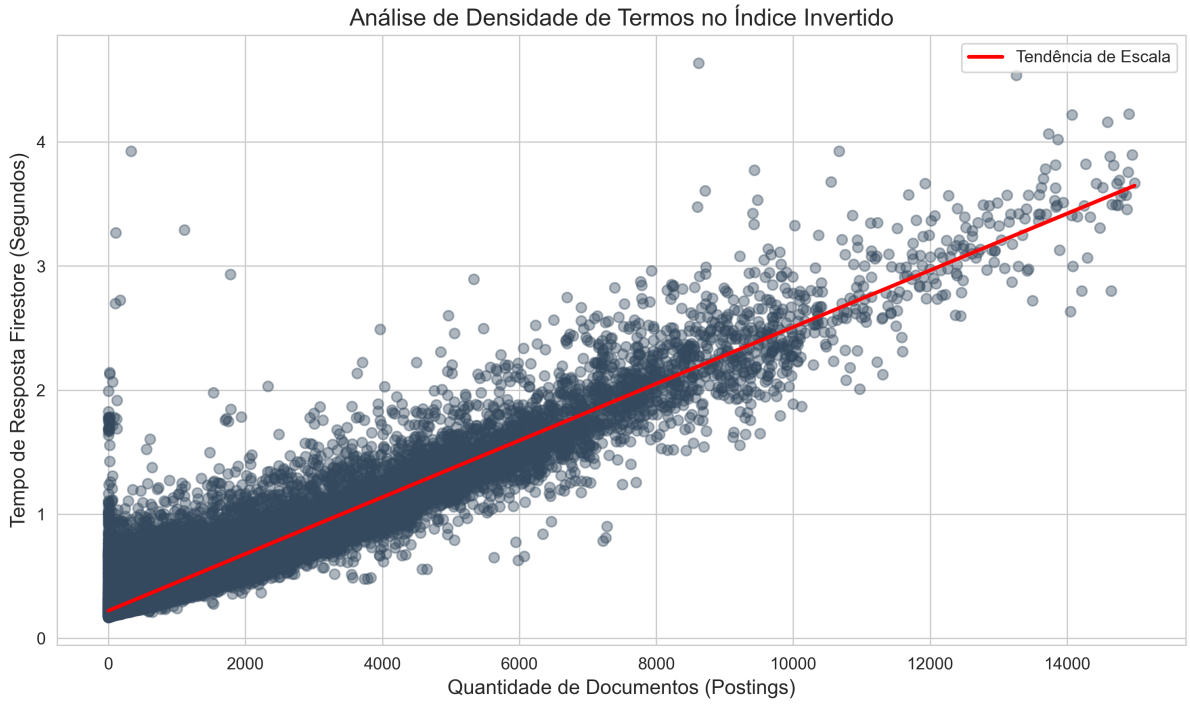
A execução bem-sucedida dessa rotina demonstrou que a infraestrutura configurada é capaz de suportar e organizar a alta densidade do índice invertido otimizado. A consolidação dessa etapa de persistência atua como o pré-requisito fundamental para a arquitetura da solução, garantindo a viabilidade técnica para a futura implementação da API em *FastAPI*, que consumirá esses dados estruturados para a efetiva aplicação do modelo TF-IDF e cálculo da Similaridade do Cosseno na classificação de ameaças em tempo real.

5.2 Validação da Persistência e Densidade do Índice

A validação da carga de dados no ambiente *cloud* ocorreu por meio da monitorização do processo de ingestão do índice invertido. É fundamental destacar que esta etapa consiste em um processo *offline* de escrita em lote (*batch upload*) para popular o banco de dados. Dessa forma, os tempos de resposta registrados são referentes exclusivamente às transações de gravação no *Google Firestore*, não possuindo relação direta com a latência de leitura da API, cuja meta de desempenho permanece inferior a 500 milissegundos.

Para garantir a estabilidade operacional e respeitar os limites de armazenamento e de 20.000 escritas diárias do plano gratuito do Google Firebase, a migração inicial foi limitada a um vocabulário de 19.500 termos únicos. É importante distinguir que, enquanto o teto de 15.000 postings filtra a densidade interna de cada termo para evitar documentos pesados no Firestore, o limite de 19.500 termos define a amplitude total do dicionário persistido nesta etapa experimental. Esta estratégia permite que o sistema opere dentro das quotas de serviço sem comprometer a eficácia do Modelo Vetorial.

Figura 4 – Distribuição da carga de dados e análise de densidade do índice no Firestore (*Processo de Escrita/Upload*)



Fonte: Dados da Pesquisa (2026).

A dispersão dos dados revela a heterogeneidade do *corpus*. É possível observar uma elevada concentração de termos técnicos com listas de documentos reduzidas, o que favorece a seletividade do Modelo Vetorial. A linha de tendência de latência apresenta uma inclinação próxima à estabilidade, indicando que o tempo de gravação não sofre degradação linear proporcional ao tamanho do objeto. A maioria das operações foi concluída em menos de 2 segundos, confirmando a robustez da infraestrutura em processar cargas de diferentes magnitudes. Esta etapa preparatória assegura a disponibilidade do dicionário no *backend*, habilitando o sistema para as operações de similaridade via API com rotinas de leitura de baixa latência.

6 RESULTADOS ESPERADOS

O projeto possui como meta principal a entrega de uma arquitetura funcional para a detecção de ameaças de *phishing* em formato de extensão para o navegador *Google Chrome*. O foco reside na validação da proposta colaborativa e na capacidade de resposta do sistema. Os indicadores definidos para a análise dos resultados contemplam:

1. **Taxa de Detecção Colaborativa:** Observação da percentagem de links identificados pela comunidade e a prontidão da disponibilidade destes dados no *Google Firestore*.

2. **Índice de Confiabilidade do Feedback:** Avaliação da maturidade da base de dados através da proporção entre reportes confirmados e alertas que apresentaram inconsistências.

3. **Funcionalidade do Modelo Vetorial:** Verificação da capacidade da *Similaridade do Cosseno* no processamento do conteúdo das páginas e no retorno de um ranqueamento de termos coerente.

4. **Desempenho da Interface:** Validação da transparência da proteção para o utilizador, com o intuito de manter o tempo médio de resposta da API abaixo de 500 milissegundos.

5. **Distribuição Geográfica:** Mapeamento relativo das ameaças através do endereço IP, com o intuito de identificar a concentração de ataques por região.

7 CRONOGRAMA

Para viabilizar o desenvolvimento da arquitetura proposta e a condução da pesquisa experimental, as atividades foram organizadas em etapas sequenciais, alinhadas aos marcos avaliativos do semestre letivo. A Tabela 1 detalha a distribuição das tarefas ao longo dos meses de execução do projeto.

Tabela 1 – Cronograma de atividades do Trabalho de Conclusão de Curso

Atividades	**Fev**	**Mar**	**Abr**	**Mai**	**Jun**	**Jul**
Definição da arquitetura e elaboração da Proposta (Entrega 1)	X	X				
Desenvolvimento da API (*Back-end*) - MVP para 13/03	X	X				
Integração em nuvem e Extensão Manifest V3 (*Front-end*)		X	X			
Testes experimentais e Entrega Intermediária (Entrega 2)			X	X		
Instrumentação, coleta de dados e refinamento do algoritmo				X	X	
Elaboração da apresentação (Slides)				X	X	
Redação final do artigo (Entrega 3)				X	X	
Defesa perante a banca examinadora					X	X

Fonte: Dados da Pesquisa (2026).

REFERÊNCIAS

APWG. **Phishing Activity Trends Report: 4th Quarter 2024**. [S.l.], 2025. Acesso em: 19 fev. 2026.

BAEZA-YATES, R. *et al.* Modern information retrieval. 07 1999.

Federal Bureau of Investigation. **Internet Crime Report 2024**. Washington, D.C., 2024. Acesso em: 19 fev. 2026.

FETTE, I.; SADEH, N.; TOMASIC, A. Learning to detect phishing emails. In: **Proceedings of the 16th international conference on World Wide Web**. [S.l.: s.n.], 2007. p. 649–656.

Google. **Lista de verificação de migração do Manifesto V3**. 2024. <https://developer.chrome.com/docs/extensions/develop/migrate/checklist?hl=pt-br>. Acesso em: 25 de fevereiro de 2026.

IBM. **Cost of a Data Breach Report 2025**. Nova York, 2025. Acesso em: 19 fev. 2026. Disponível em: <https://www.ibm.com/reports/data-breach>.

KOLESNIKOV, A.; CUOZZO, L. Análisis del ciclo de vida de las páginas de phishing y de estafas. **Securelist by Kaspersky**, 2021. Acesso em: 19 fev. 2026. Disponível em: <https://securelist.lat/phishing-page-life-cycle/95951/>.

MARTíNEZ, G. Phishing: Three key trends and how to protect yourself. **LACNIC Blog**, mar. 2025. Acesso em: 19 fev. 2026. Disponível em: <https://blog.lacnic.net/en/cybersecurity-smishing-attacks/>.

NIELSEN, J. **10 Usability Heuristics for User Interface Design**. Nielsen Norman Group, 1994. Disponível em: <https://www.nngroup.com/articles/ten-usability-heuristics/>. Acesso em: 20 fev. 2026.

Verizon. **2024 Data Breach Investigations Report (DBIR)**. Nova York, 2024. Acesso em: 19 fev. 2026. Disponível em: <https://www.verizon.com/business/resources/reports/dbir/>.

ZHANG, Y.; HONG, J. I.; CRANOR, L. F. Cantina: a content-based approach to detecting phishing web sites. In: **Proceedings of the 16th International Conference on World Wide Web**. New York, NY, USA: Association for Computing Machinery, 2007. (WWW '07), p. 639–648. ISBN 9781595936547. Disponível em: <https://doi.org/10.1145/1242572.1242659>.